

Assignment Report: Handwritten Digit Recognition using Neural Network

1. Introduction

The objective of this assignment is to develop an end-to-end machine learning pipeline capable of recognizing handwritten digits. The project consists of two primary components: a model training script that utilizes a Neural Network to extract patterns from a standard dataset, and an interactive Graphical User Interface (GUI) that allows a user to draw a digit and receive real-time predictions.

2. Methodology: Data Processing and Model Training (`train_model.py`)

The foundational logic relies on training a classification model to recognize spatial patterns in pixel data.

- **Dataset Acquisition:** The model utilizes the standard **MNIST dataset**, which contains grayscale images of handwritten digits (0-9).
- **Data Preprocessing**
 - **Dimensionality:** Raw images are 28 x 28 pixel matrices. For processing, each image is flattened into a 1-dimensional array of 784 features.
 - **Normalization:** Pixel intensity values (0–255) are divided by 255 to scale them to a [0, 1] range. This ensures numerical stability during the neural network's backpropagation process.
- **Model Selection and Architecture:** The project utilizes a Multi-Layer Perceptron (MLP).
 - **Input Layer:** 784 neurons corresponding to the flattened pixel data.
 - **Hidden Layers:** Dense layers using the ReLU (Rectified Linear Unit) activation function to capture non-linear relationships within the handwriting data.
 - **Output Layer:** 10 neurons with a Softmax activation function, which provides a probability distribution across the digits 0 through 9.
- **Model Persistence:** Once trained, the model's weights and architecture are saved to disk (e.g., `mnist_model.keras`). This allows the application to perform inference later without needing to retrain the network.

3. Implementation: The Interactive GUI (`digit_gui.py`)

To make the model accessible, a custom graphical interface was developed using the Tkinter library. The interface ensures user-generated data mimics the format of the training data.

- **Data Capture:** The user draws on a 280 x 280 pixel canvas with a black background and white brush.
- **Real-time Transformation Pipeline:** When the "Predict" button is clicked, the application applies a strict preprocessing pipeline to the user's drawing:
 1. **Downsampling:** The high-resolution canvas is resized to the 28 x 28 dimension expected by the network.

2. **Array Conversion & Scaling:** The image is converted to a NumPy array and normalized by 255.
3. **Reshaping:** The 2D array is flattened into a 1 x 784 vector.
- **Inference:** This vector is passed to the pre-loaded Neural Network. The model identifies the neuron in the output layer with the highest activation value, representing the most probable digit, which is then displayed to the user.

4. Test Cases

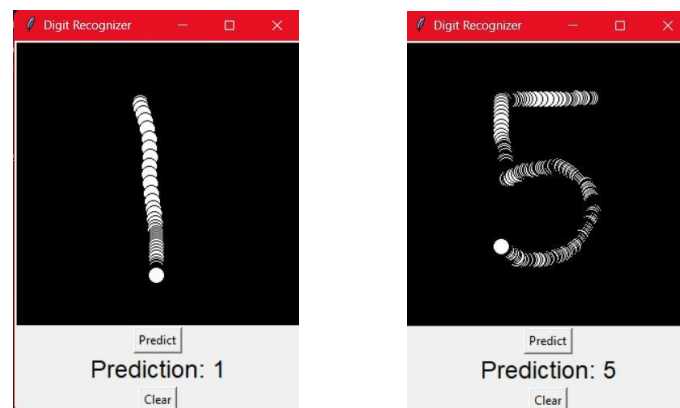


Figure 1: Successful Classification Case

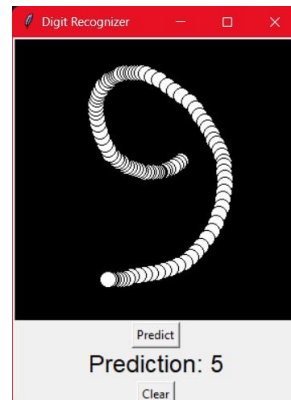


Figure 2: Misclassification Analysis

In Figure 1, the network correctly identifies clear strokes by mapping them to high-probability output neurons.

As seen in Figure 2, errors often occur due to non-standard handwriting styles or "noise" created during the downsampling process. Unlike simpler models, the neural network's accuracy is heavily dependent on the depth of the hidden layers and the variety of the training subset used.